

ArnoTek Arduino Utilities – HeartBeat

Arduinos have an onboard LED that is connected to pin 13 and this makes a nice heartbeat indicator that the Arduino is running as it should and that it is not stuck in an infinite loop, excessive delay, or is in sleep mode. Also, depending on your application, sometimes it is necessary to invoke code at particular time intervals.

These capabilities are what this heartbeat utility addresses.

To use this utility, install the ArnoTek_HeartBeat.h file in your Arduino libraries directory. This will make the utility accessible to all of your projects. ArnoTek utilities and global variables are prefixed with “ArnoTek” in an effort to prevent conflicts with include files from other developers.

Let’s start with the simplest implementation – a heartbeat indicator.

ArnoTek Arduino Utilities – HeartBeat

Example 1

Anytime you wish to use this utility, you must tell the compiler to include the .h file in your application. This is accomplished by placing the **#include <ArnoTek_HeartBeat.h>** compiler directive at the beginning of your code.

The next thing to do is to define the object you wish to use for the heartbeat by specifying a constructor. This is place before the setup and loop functions so all of the code has access to your object.

The HeartBeat class supports a number of different constructors. This example defines your object and the timeout value that you wish to toggle the LED connected to default pin 13. This constructor supports one parameter that is the interval in milliseconds that you wish to toggle the LED. This parameter is a long unsigned variable and it may be a value from 1 millisecond to 4,294,967,295 milliseconds which is 49 days, 17 hours, 2 minutes, 47 seconds, and 296 milliseconds.

ArnoTek_HeartBeat HeartBeat(static_cast<long unsigned>(500));

- “ArnoTek_HeartBeat” is the name of the class.
- “HeartBeat” is the name of the object for your code. You may name this anything that you wish.
- The parameter being passed to the constructor is cast as a “long unsigned” value using the “static_cast<long unsigned>()” syntax.
- The value of 500 indicates that the LED is to toggle on and off every 500 milliseconds or half a second.

Now that your object has been defined, you only need to place one line of code in your loop function.

ArnoTek Arduino Utilities – HeartBeat

HeartBeat.Toggle();

That is it. Every time you process the loop function, the Toggle function determines if the specified number of milliseconds has elapsed and if it has, the LED is toggled on. After the specified number of milliseconds has passed a second time, the LED is toggled off and so on. Thus, the LED toggles on and off at half second intervals.

As you can see, this is a very easy way to add a heartbeat indicator to your code.

ArnoTek Arduino Utilities – HeartBeat

Example 2

But wait, what if you want an additional heartbeat that toggles a different pin for some external component?

You may define as many heartbeat objects as you wish. In this case we will define a second heartbeat object. Say we wish to toggle pin 12 every minute to provide a signal to an external component. The external component would be connected to pin 12 of the Arduino and the object would be defined as

```
ArnoTek_HeartBeat Pin12_signal(static_cast<long unsigned>(60000), 12);
```

This constructor supports two parameters, one for the millisecond interval and the second defines the pin to toggle.

Pin12_signal is the name of the object and you may name this anything that you wish. 60000 is the number of milliseconds in a minute and as before, it must be cast as a long unsigned value. The second parameter indicates the pin to toggle and in this case it is pin 12 but you may use any pin that you wish. The code in the loop function is identical and in this case is:

```
HeartBeat.Toggle();
```

```
Pin12_signal.Toggle();
```

So in this case, the LED at pin 13 toggles every half a second, and pin 12 is toggled every minute. Since the HeartBeat object and the Pin12_signal object are different objects, they are completely independent of each other.

ArnoTek Arduino Utilities – HeartBeat

Example 3

Ok, that is all well and good, but what if you don't want to toggle a pin but instead, you want to call a function in your code on a repeating basis on a specified time interval? This class supports that capability with a constructor that has a parameter for the time duration and a second parameter that contains the address of the function you wish to be called when the time duration has passed, as follows.

```
ArnoTek_HeartBeat Seconds90(static_cast<long unsigned>(90000),  
                           &My90SecFunction);
```

As before, the first parameter to the constructor is a long unsigned value and in this case specifies a time duration value of 90,000 milliseconds, or 90 seconds. The second parameter specifies the address to the function that is to be called each time the duration passes. No code is needed in the **setup** function and the **loop** function as follows:

// Instantiate the timer objects

```
ArnoTek_HeartBeat HeartBeat(static_cast<long unsigned>(500));  
ArnoTek_HeartBeat Pin12_signal(static_cast<long unsigned>(60000), 12);  
ArnoTek_HeartBeat Seconds90(static_cast<long unsigned>(90000)  
                             , &My90SecFunction);
```

void loop

```
{  
    HeartBeat.Toggle();           // heartbeat LED  
    Pin12_signal.Toggle();        // Toggle pin 12 every minute  
    Seconds90.Toggle();           // Call function specified earlier every 90 seconds  
}
```

void My90SecFunction_1(long unsigned currentTimer)

```
{  
    // This code will run once every 90 seconds. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
}
```

ArnoTek Arduino Utilities – HeartBeat

Example 4

What about a case that is similar to what we already discussed, but now you want another function to run every 5 minutes in addition to the 90 second function? Basically, all we have to do is add an additional instantiation of the HeartBeat object and set its duration to 5 minutes and give it the address of the function that is to be processed every 5 minutes.

// Instantiate the timer objects

```
ArnoTek_HeartBeat HeartBeat(static_cast<long unsigned>(500));  
ArnoTek_HeartBeat Pin12_signal(static_cast<long unsigned>(60000), 12);  
ArnoTek_HeartBeat Seconds90(static_cast<long unsigned>(90000)  
                                , & My90SecFunction);  
ArnoTek_HeartBeat Minutes5(static_cast<long unsigned>(300000)  
                                , &My5MinFunction);
```

void loop

```
{  
    HeartBeat.Toggle();           // heartbeat LED  
    Pin12_signal.Toggle();       // Toggle pin 12 every minute  
    Seconds90.Toggle();          // Call the function target every 90 seconds  
    Minutes5.Toggle();           // Call the function target once every 5 minutes  
  
}
```

void My90SecFunction(long unsigned myCurrentTimer)

```
{  
    // This code will run once every 90 seconds. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
  
}
```

void My5MinFunction(long unsigned myCurrentTimer)

```
{  
    // This code will run once every 5 minutes. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
  
}
```

ArnoTek Arduino Utilities – HeartBeat

Example 5

What about a case that is close to what we already discussed, but now you want a routine to run every 5 minutes but you only want the 90 second function to run only once?

Basically, all we have to do is add an additional instantiation of the HeartBeat object and set its duration to 5 minutes and give it the address of the function that is to be processed every 5 minutes.

However in the Seconds90 routine, it needs to be disabled so it won't be called again. That can be done by setting the Callback modifier and setting it to a **nullptr** value as follows.

Note that the HeartBeat code will still be checking if there is a callback function every 90 seconds. To reduce this overhead, it is advisable to also set the timer delay to a very high value.

// Instantiate the timer objects

```
ArnoTek_HeartBeat HeartBeat(static_cast<long unsigned>(500));
ArnoTek_HeartBeat Pin12_signal(static_cast<long unsigned>(60000), 12);
ArnoTek_HeartBeat Seconds90(static_cast<long unsigned>(90000)
                             , &My90SecFunction);
ArnoTek_HeartBeat Minutes5(static_cast<long unsigned>(300000)
                             , &My5MinFunction);
```

void loop

```
{
    HeartBeat.Toggle();           // heartbeat LED
    Pin12_signal.Toggle();        // Toggle pin 12 every minute
    Seconds90.Toggle();           // Call the function target every 90 seconds
    Minutes5.Toggle();            // Call the function target once every 5 minutes
}
```

ArnoTek Arduino Utilities – HeartBeat

```
void My90SecFunction(long unsigned myCurrentTimer)
{
    // This code will run once every 90 seconds. The input parameter currentTimer
    // contains the current value of the millis() timer when this function is called.

    Seconds90.CallBack(nullptr);    // Disable callback
    Seconds90.Delay(4294967000);    // Set the timer delay to a very high number
}
```

```
void My5MinFunction(long unsigned myCurrentTimer)
{
    // This code will run once every 5 minutes. The input parameter currentTimer
    // contains the current value of the millis() timer when this function is called.
}
```

ArnoTek Arduino Utilities – HeartBeat

Example 6

Besides the multiple constructors to support these capabilities, there is also a series of Init functions that are typically placed in the setup function. These are normally used instead of the constructors with parameters. They are used as follows.

Each of the desired heartbeat objects must be instantiated as before but this time with no parameters.

```
// Instantiate the timer objects  
ArnoTek_HeartBeat HeartBeat;  
ArnoTek_HeartBeat Pin12_signal;  
ArnoTek_HeartBeat Seconds90;  
ArnoTek_HeartBeat Minutes5;
```

Each of the HeartBeat objects must then be initialized in the setup function. Note that the Init functions do not

```
void setup  
{  
    HeartBeat.Init(static_cast<long unsigned>(500));  
    Pin12_signal.Init(static_cast<long unsigned>(60000), 12);  
    Seconds90.Init(static_cast<long unsigned>(90000), &My90SecFunction);  
    Minutes5.Init(static_cast<long unsigned>(300000), &My5MinFunction);  
}
```

The loop function and the callback functions are exactly as before.

```
void loop  
{  
    HeartBeat.Toggle();           // heartbeat LED  
    Pin12_signal.Toggle();       // Toggle pin 12 every minute  
    Seconds90.Toggle();         // Call the function target every 90 seconds  
    Minutes5.Toggle();          // Call the function target every 5 minutes  
}
```

ArnoTek Arduino Utilities – HeartBeat

```
void My90SecFunction (long unsigned myCurrentTimer)  
{  
    // This code will run once every 90 seconds. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
  
    Seconds90.CallBack = nullptr; // Disable callback  
    Seconds90.Delay = 4294967000; // Set the timer delay to a very high number  
}
```

```
void My5MinFunction (long unsigned myCurrentTimer)  
{  
    // This code will run once every 5 minutes. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
}
```

ArnoTek Arduino Utilities – HeartBeat

Example 7

But why bother with using the Init functions when you can accomplish the same thing with the constructors? The Init functions also can provide an additional parameter for the initial **millis()** value. This enables all of the HeartBeat objects to use the same initial time value. The Toggle function also supports specifying the timer value. This provides the capability for all of the HeartBeat objects to use the same timer value for each of their processing. For example:

```
// Instantiate the timer objects  
ArnoTek_HeartBeat HeartBeat;  
ArnoTek_HeartBeat Pin12_signal;  
ArnoTek_HeartBeat Seconds90;  
ArnoTek_HeartBeat Minutes5;
```

Each of the HeartBeat objects must then be initialized in the setup function. Note that the Init functions do not

```
void setup  
{  
  long unsigned initialTimer = millis();  
  HeartBeat.Init(initialTimer, static_cast<long unsigned>(500));  
  Pin12_signal.Init(initialTimer, static_cast<long unsigned>(60000), 12);  
  Seconds90.Init(initialTimer, static_cast<long unsigned>(90000)  
    , &My90SecFunction);  
  Minutes5.Init(initialTimer, static_cast<long unsigned>(300000)  
    , &My5MinFunction);  
}
```

The loop function and the callback functions are exactly as before but with one additional parameter.

```
void loop  
{  
  long unsigned currentTimer = millis();  
  
  HeartBeat.Toggle(currentTimer); // heartbeat LED  
  Pin12_signal.Toggle(currentTimer); // Toggle pin 12 every minute  
  Seconds90.Toggle(currentTimer); // Call the function target every 90 seconds  
  Minutes5.Toggle(currentTimer); // Call the function target every 5 minutes  
}
```

ArnoTek Arduino Utilities – HeartBeat

```
void My90SecFunction(long unsigned myCurrentTimer)  
{  
    // This code will run once every 90 seconds. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
  
    Seconds90.CallBack = nullptr; // Disable callback  
    Seconds90.Delay = 4294967000; // Set the timer delay to a very high number  
}
```

```
void My5MinFunction _5(long unsigned myCurrentTimer)  
{  
    // This code will run once every 5 minutes. The input parameter currentTimer  
    // contains the current value of the millis() timer when this function is called.  
}
```

The HeartBeat class also has modifiers to change the pin number where a value of 0 indicates not toggle any pins. Note that there is no verification that the specified parameter is a valid pin number.